Part B: review of the script, with suggested fixes

- Thank you for sharing the script. I ran it unchanged on the 2012-2021 Nifty data (the locked 2022-2025 period was not used in this part), and it prints a Sharpe of **13.7**.
- The original report said "above 4", so the file or seed may have differed, but the points below come from the code itself and do not depend on the data.
- `compare_fixes.py` applies the suggested fixes one at a time, each step keeping the earlier ones, and the numbers quoted below come from that run (`outputs/markdown/ladder.md`, also as `outputs/pdf/ladder.pdf`).

step	what changed	Sharpe	direction accuracy
0	original script	13.70	1.000
1	take `ret` out of the features	2.85	0.612
2	predict tomorrow and earn tomorrow (random split still on)	0.55	0.525
3	split by time instead of shuffling	1.01	0.538
4	volume z-score from the past only	0.56	0.535
5a	trade at next open	0.72	0.530
5b	0.05% per side costs	0.27	0.530
6a	walk-forward retraining, shallow forest	0.21	0.514
6b	same, after costs (the corrected script)	-0.00	0.514

- Steps 3 to 6 all sit inside the noise, because one test split of about 1,000 days has a Sharpe standard error of roughly 0.5.
- The small rise at step 3 therefore comes from a different test sample and is not a fix helping.

Points to address, in order of impact

1. The label is also one of the features

- The target is `ret > 0` and `ret` is in `X`, so the forest can simply learn "if ret is positive, predict 1". Test accuracy is exactly 100%, and removing `ret` alone takes the Sharpe from 13.7 to 2.85.
- *Suggested fix:* remove `ret` from the same-day features, or (better, see point 2) predict the following day.

2. The features already contain today's close

- `dist_ma` compares today's close with its 20-day average, so a big up day pushes it up by construction, and `vol_20` also includes today's return.
- The model is therefore predicting today's return from a function of today's return, and accuracy is still 61% after step 1 where about 50 to 54% would be expected.
- *Suggested fix:* predict tomorrow's return from what is known at today's close.

3. The trade cannot be executed as written

- The script trades "at today's close using today's signal" and then multiplies the position by today's return. Today's close is known only after the market shuts, so today's close-to-close move cannot be earned.
- *Suggested fix:* follow the brief by filling at the next open and exiting at the open after (`fwd_ret` in the corrected script).

4. The random `train_test_split` is not well suited to a time series

- Shuffling puts, for example, 2015 days in training right next to 2015 days in the test set. Rolling features share 19 of 20 days between neighbouring rows and returns are autocorrelated, so the training set effectively contains information about the test answers.
- It also trains on the future and tests on the past, which never happens in live trading.
- *Suggested fix:* train only on dates before the test block, leave a gap of 2 days (the label looks 2 opens ahead) and retrain as time moves on (walk-forward). If K-fold is used, add purging and an embargo.

5. The volume z-score uses the whole sample

- `df["volume"].mean()` and `.std()` include the test period, so each row is standardised with information from the future. Volume also trends upward, so the feature can act as a proxy for the calendar.
- Volume is zero throughout 2012, which is missing data in Yahoo and not "no trading".
- *Suggested fix:* treat zeros as missing and use the mean and standard deviation of the past 60 days only.

6. Costs are not included

- The signal flips between long and short on many days, and with 0.05% per side one flip costs 0.10%. In the ladder, costs alone took the step 5 model from a Sharpe of 0.72 to 0.27.
- Even the corrected walk-forward model changes position about 77 times a year, which costs it 3.8% of capital a year.
- *Suggested fix:* deduct 0.05% per side on every change of position.

7. The Sharpe is computed on a random 30% of the days

- `signal` is NaN on the other 70%, so `strat_ret` exists only on scattered days, and `mean()/std()*sqrt(252)` treats them as a normal daily series. No equity curve or drawdown can be built from that, and it is not the return of any strategy that could be run live.
- *Suggested fix:* use one contiguous out-of-sample stretch where the strategy trades every day.

8. The forest has no limit on how closely it can fit the noise

- `RandomForestClassifier(n_estimators=500)` has unlimited depth and a leaf size of 1, so with about 2,400 noisy rows it can fit the noise very closely.
- *Suggested fix:* limit its capacity. I used `max_depth=3, min_samples_leaf=50`.

9. A single split gives one number with no sense of uncertainty

- A Sharpe from one test set of about 1,000 days is very noisy, with a standard error around 0.5.
- *Suggested fix:* report a confidence interval. The corrected script uses a block bootstrap, and for the corrected run it is -0.88 to 0.94, which is consistent with no edge at all.

10. Smaller points

- The model has no `random_state`, so results change from run to run, and `read_csv` is not followed by a sort on the date, so an unsorted file would make the rolling windows run backwards.
- Hard 0/1 predictions mean the position is always fully long or fully short whatever the confidence. A probability threshold or position sizing would be sensible, although I kept it simple.
- The Sharpe uses a risk-free rate of zero, which is acceptable for long/short futures, but a long-only strategy would normally subtract roughly 6% (Indian T-bills).

- Direction accuracy ignores the size of the move, so a model that is right on small days and wrong on big days can lose money even at 55% accuracy.

What Sharpe would I expect after the fixes?

- Roughly **0**, and in my view not above 0.5. On the research data the corrected pipeline gives 0.21 before costs and 0.00 after, with direction accuracy of 51.4%.
- Always guessing "up" gives 54.4%, because the next-open-to-open return was positive on about 54% of the test days.
- Daily index direction has very little signal in price, volatility and volume features, so after leakage is removed what remains is mostly noise, less costs. If a range is needed, I would say -0.5 to +0.5, centred on zero.
- The one real effect I found in Part C had a research Sharpe of 1.25 and about 0.8 out of sample, which is similar to buy and hold, and it came after a careful search and not from a random forest on four features.

Files

- `original_script.py`: the original script, unchanged
- `corrected_script.py`: my suggested version (run it with `nifty_daily.csv` in the same folder)
- `compare_fixes.py`: applies the fixes one at a time and prints the table above
- `nifty_daily.csv`: Nifty 50 daily bars 2012-2021 from Yahoo Finance (research period only)